

MapReduce Combiner and Multi-Step MRJob

Lab Objective

Build on the one-step MapReduce programs from Lab 01 by introducing the Combiner optimization and multi-step MapReduce pipelines using MRJob.

Lab Outcome

After completing this lab successfully, students will be able to:

1. Explain the role and purpose of the Combiner in the MapReduce pipeline.
2. Identify when it is safe (and unsafe) to use a Combiner.
3. Implement a Combiner class in MRJob for local pre-aggregation.
4. Design and implement multi-step MapReduce jobs using the steps() method.
5. Chain MapReduce phases to solve problems requiring multiple passes over data.
6. Implement a MapReduce-based join operation to combine data from two sources.

Psychomotor Learning Levels

This lab involves activities that encompass the following learning levels in psychomotor domain.

Level	Category	Meaning	Keywords
P2	Manipulation	Reproduce activity from instruction or memory.	Copy, response, trace, Show, Start, Perform, Execute, Recreate.
P3	Precision	Execute skill reliably, independent of help.	Demonstrate, Complete, Show, Perfect, Calibrate, Control.
P4	Articulation	Adapt and integrate skills to satisfy a complex requirement.	Construct, Solve, Combine, Coordinate, Integrate, Adapt.

Lab Activities

1. Environment Setup (Re-run at Every Session)

Colab sessions reset between sessions. Run all setup cells from Lab 01 before proceeding. As a quick reference, the essential cells are:

```
!wget https://downloads.apache.org/hadoop/common/hadoop-3.4.1/hadoop-3.4.1.tar.gz
!tar -xzf hadoop-3.4.1.tar.gz && cp -r hadoop-3.4.1/ /usr/local/
JAVA_HOME = !readlink -f /usr/bin/java | sed "s:bin/java::"
java_home_text = JAVA_HOME[0]
!echo export JAVA_HOME=$java_home_text >> /usr/local/hadoop-3.4.1/etc/hadoop/hadoop-
env.sh
import os
os.environ['HADOOP_HOME'] = "/usr/local/hadoop-3.4.1"
os.environ['JAVA_HOME'] = java_home_text
!pip install mrjob
!/usr/local/hadoop-3.4.1/bin/hadoop fs -mkdir file:///tmp
!/usr/local/hadoop-3.4.1/bin/hadoop fs -chmod 777 file:///tmp
```

Note: Always re-run setup before starting tasks. If you see errors like 'hadoop: command not found' or 'No module named mrjob', your session has reset.

2. Understanding the Combiner

In a standard MapReduce job, every (key, value) pair produced by the Mapper is sent across the network to the Reducer. For large datasets, this network transfer — called the Shuffle — is the primary performance bottleneck. A Combiner is an optional local aggregation step that runs on each node immediately after the Map phase, before data is sent over the network. It acts as a mini-Reducer, collapsing many small values into a single aggregated value per key, drastically reducing shuffle traffic.

Data flow WITHOUT a Combiner:

```
Input --> [MAPPER] --> (many key-value pairs) --> Shuffle --> [REDUCER] --> Output
```

Data flow WITH a Combiner:

```
Input --> [MAPPER] --> [COMBINER (local)] --> (fewer pairs) --> Shuffle --> [REDUCER] --> Output
```

A Combiner can only be used when the aggregation operation is both commutative (order does not matter) and associative (grouping does not matter). The table below summarizes common cases:

Operation	Commutative?	Associative?	Combiner Safe?
Sum / Count	Yes	Yes	YES
Maximum / Minimum	Yes	Yes	YES
Average (direct)	Yes	NO	NO — use sum+count pattern
Concatenation	No	No	NO

Note: NEVER use a Combiner for direct averaging. If the combiner computes partial averages, the final reducer averaging partial averages gives the WRONG result. Instead, emit (sum, count) tuples and compute the average only in the final Reducer.

3. Lab Task 01: Average Rating per Movie (With Combiner)

Compute the average rating for each movie in the MovieLens dataset. This requires a Combiner that correctly handles partial sums — the (sum, count) pattern.

Algorithm:

- Mapper: parse each line and emit (movieID, rating).
- Combiner: for each movieID on the local node, accumulate (partial_sum, partial_count) and emit (movieID, (partial_sum, partial_count)).
- Reducer: receive all (partial_sum, partial_count) tuples for a movieID, sum them, and divide to get the final average.

```
%%writefile AverageRating.py
# import modules
from mrjob.job import MRJob
from mrjob.step import MRStep
```

```

class AverageRating(MRJob):

    def steps(self):
        return [
            MRStep(mapper=self.mapper,
                    combiner=self.combiner,
                    reducer=self.reducer)
        ]

    # Mapper: emit (movieID, rating) for each record
    def mapper(self, _, line):
        (userID, movieID, rating, timestamp) = line.split('\t')
        yield movieID, float(rating)

    # Combiner: local pre-aggregation using sum+count pattern
    # This is SAFE because we never compute partial averages
    def combiner(self, key, values):
        total = 0
        count = 0
        for value in values:
            total = total + value
            count = count + 1
        yield key, (total, count)

    # Reducer: receive (partial_sum, partial_count) tuples
    # sum them and compute final average
    def reducer(self, key, values):
        total = 0
        count = 0
        for r, c in values:
            total = total + r
            count = count + c
        yield key, total / count

if __name__ == '__main__':
    AverageRating.run()

```

```

!python AverageRating.py /content/ml-100k_folder/ml-100k/u.data > result_Task1.txt
!cat result_Task1.txt | head -20

```

4. Lab Task 02: Top 3 Highest-Rated Movies (Multi-Step MRJob)

Find the 3 movies with the highest average ratings. This cannot be done in one step because Step 1 computes all average ratings (a many-to-many transformation), and Step 2 selects the top 3 (a many-to-one global selection). Each step must complete fully before the next can begin.

Steps() method in MRJob:

The steps() method returns a list of MRStep objects. Each MRStep specifies the mapper, optional combiner, and reducer for that phase:

```

def steps(self):
    return [
        MRStep(mapper=self.step1_mapper,    # Step 1
                combiner=self.step1_combiner,
                reducer=self.step1_reducer),
        MRStep(mapper=self.step2_mapper,    # Step 2
                reducer=self.step2_reducer),
    ]

```

Key pattern — using a single dummy key:

In Step 2, the Mapper emits None (or any constant) as the key. This forces all (average, movieID) pairs from every node to route to a single Reducer, which can then find the global Top 3. Without a common key, different Reducers would find local Top 3s, not the global Top 3.

```
%%writefile Top3RatedMovies.py
# import modules
from mrjob.job import MRJob
from mrjob.step import MRStep

class Top3RatedMovies(MRJob):

    def steps(self):
        return [
            MRStep(mapper=self.mapper,
                    combiner=self.combiner,
                    reducer=self.reducer),
            MRStep(mapper=self.mapper1,
                    reducer=self.reducer1)
        ]

    # STEP 1 - Compute average rating per movie
    # Mapper: emit (movieID, rating)
    def mapper(self, _, line):
        (userID, movieID, rating, timestamp) = line.split('\t')
        yield movieID, float(rating)

    # Combiner: local pre-aggregation (sum+count pattern)
    def combiner(self, key, values):
        total = 0
        count = 0
        for value in values:
            total = total + value
            count = count + 1
        yield key, (total, count)

    # Reducer: compute final average per movieID
    def reducer(self, key, values):
        total = 0
        count = 0
        for r, c in values:
            total = total + r
            count = count + c
        yield key, total / count

    # STEP 2 - Find Top 3 movies by average rating
    # Mapper: route ALL pairs to a single reducer using constant key 'X'
    # (average, movieID) tuple: average first so Python's sort works correctly
    def mapper1(self, key, value):
        yield "X", (value, key)

    # Reducer: sort all pairs descending and yield Top 3
    def reducer1(self, key, values):
        top3 = sorted(values, reverse=True)[0:3]
        for v in top3:
            yield v[1], v[0] # emit (movieID, average_rating)

if __name__ == '__main__':
    Top3RatedMovies.run()
```

```
!python Top3RatedMovies.py /content/ml-100k_folder/ml-100k/u.data > result_Task3.txt
!cat result_Task3.txt
```

5. Lab Task 03: MapReduce Join Operation

In real-world big data pipelines, data often comes from multiple sources (like a relational database join). MapReduce can perform joins by using the join key as the Mapper's output key, routing matching records from both sources to the same Reducer.

Create the following input file. Each line is either a User record (U) or a Rating record (R):

```
U, 101, Alice
U, 102, Bob
U, 103, Charlie
U, 104, Danny
R, 101, 201, 5
R, 101, 202, 4
R, 102, 201, 3
R, 102, 202, 4
R, 102, 203, 5
R, 104, 203, 2
```

Column descriptions:

- User record (U): U, userID, userName
- Rating record (R): R, userID, movieID, rating

Goal: join users with their ratings to produce output lines of the form: (userID) -> (userName, (movieID, rating))

Step — Create the Input File

```
input_data = ""
U, 101, Alice
U, 102, Bob
U, 103, Charlie
U, 104, Danny
R, 101, 201, 5
R, 101, 202, 4
R, 102, 201, 3
R, 102, 202, 4
R, 102, 203, 5
R, 104, 203, 2

with open('input_user_rating.txt', 'w') as f:
    f.write(input_data)
print('input_user_rating.txt created')
```

```
%%writefile JoinOperation.py
# import modules
from mrjob.job import MRJob
from mrjob.step import MRStep

class JoinOperation(MRJob):

    def steps(self):
        return [
            MRStep(mapper=self.mapper,
                    reducer=self.reducer)
        ]

    # Mapper: use userID as the key for both record types
    # This routes User and Rating records with the same userID
    # to the same Reducer
    def mapper(self, _, line):
        word = line.split(",")
        if word[0].strip() == 'U':
            # User record: emit (userID, ('U', userName))
```

```

        yield word[1].strip(), (word[0].strip(), word[2].strip())
    else:
        # Rating record: emit (userID, ('R', movieID, rating))
        yield word[1].strip(), (word[0].strip(), word[2].strip(), word[3].strip())

# Reducer: for each userID, match the User record with all Rating records
def reducer(self, key, values):
    user_name = None
    ratings = []

    for v in values:
        if v[0] == 'U':
            user_name = v[1]          # store the user name
        else:
            ratings.append((v[1], v[2])) # collect all (movieID, rating) pairs

    # Emit one output line per rating for this user
    for rating in ratings:
        yield key, (user_name, rating)

if __name__ == '__main__':
    JoinOperation.run()

```

```

!python JoinOperation.py input_user_rating.txt > result_Task4.txt
!cat result_Task4.txt

```

MapReduce Combiner and Multi-Step MRJob

You must write all MRJob programs using %%writefile and run them in separate cells. Save all outputs to .txt files using redirection (> output.txt).

Lab Task # 01 (Average Rating with Combiner):

1.(a). Run AverageRating.py on the full MovieLens u.data file. Save the output to result_Task1.txt. Take a screenshot of the first 20 lines of result_Task1.txt.

1.(b). Inspect the combiner() method. Answer the following questions in your submission document:

- i. Why is it incorrect to compute the average directly inside the Combiner (e.g., yield key, total/count)?
- ii. What does the Combiner emit? Write the exact format of its output key-value pair.
- iii. What does the Reducer receive as input for a given movieID? How does it differ from what the Mapper emitted?

1.(c). Modify AverageRating.py to also output only movies with an average rating greater than 4.0. Name it HighlyRatedMovies.py. Run it and save to result_Task1c.txt.

Lab Task # 02 (Top 3 Highest-Rated Movies):

2.(a). Run Top3RatedMovies.py on the MovieLens u.data file. Save the output to result_Task3.txt. Take a screenshot of the final 3 results.

2.(b). Trace the data flow between the two steps. Answer the following:

- iv. What is the output format of Step 1's Reducer? Write one example line.
- v. What does Step 2's Mapper receive as (key, value)? What does it emit?
- vi. Why is the key "X" (or None) used in mapper1()? What would happen if each movie's ID was used as the key instead?
- vii. In reducer1(), why is the tuple stored as (value, key) — average first, movieID second? What would happen if you reversed the order?

2.(c). Modify the program to find the Bottom 3 movies (lowest average ratings). Name it Bottom3RatedMovies.py. Run it and save to result_Bottom3.txt.

Lab Task # 03 (Join Operation):

3.(a). Create input_user_rating.txt and run JoinOperation.py. Save the output to result_Task4.txt. Take a screenshot of the full output.

3.(b). Answer the following:

- viii. What is the join key used by the Mapper? Why is this key chosen?

- ix. User 103 (Charlie) has no Rating records. What does the Reducer output for userID 103? Explain why.
- x. What type of SQL join does this MapReduce join implement — INNER JOIN, LEFT JOIN, or FULL OUTER JOIN? Justify your answer.

3.(c). Add a new Rating record to the input file:

```
R, 103, 204, 4
```

Re-run JoinOperation.py and confirm that Charlie now appears in the output. Take a screenshot.

Submission

Submit your completed Colab notebook (.ipynb) and a document (.docx) containing screenshots for every task and answers to all written questions.

Name your files as per the following format:

- Notebook: 2022-1-60-001_CSE488_LAB02.ipynb
- Document: 2022-1-60-001_CSE488_LAB02.docx

Submit via the submission link provided in Google Classroom before the next lab session.